Module 5 · xloader: the secure app loader

xPython framework guide, concept edition · goal → functions → data flow → connections → testing · 31 October 2026

> **In one line:** `xloader` installs and runs an app **only if the app developer signed it** and **no file has changed since**, and it runs the app **only inside the sandbox** written in its signed manifest.

1. Goal

On a normal MicroPython board, anyone can copy a `.py` file to the flash and run it, and that code can do anything: read every file, use every key, talk to the network. `xloader` is the **gatekeeper for application code**. It makes three promises:

- **Only trusted code is installed:** the package must carry a valid signature from the device's **app** trust anchor (set by `xinit`).
- **Only unmodified code runs:** the SHA-256 of every file is in the signed manifest, and it is **checked again before every run**.
- **Code runs only inside its sandbox:** the manifest's permissions become a policy that `xpolicy` enforces, plus a time limit that `xwatch` enforces.

Analogy: a sealed medicine box from a pharmacy. The **label** (manifest) lists every pill and its exact weight, and the label carries the **manufacturer's stamp** (signature). The pharmacist checks the stamp, weighs every pill against the label, and refuses the box if anything differs. And even a good medicine comes with a **dose limit** (permissions and time limit).

`xloader` is Python, **frozen into the firmware**: it is compiled into the firmware image, not stored as a file, so nobody can replace it on the filesystem. Only a new signed firmware (`xota`) can change it.

2. The app package (.xpk)

An app is shipped as one binary file. All numbers are little-endian.

```
[ "XPK1" 4 B | manifest length 4 B | manifest (JSON) | signature 64 B | file entries ... ]
  file entry = [ name length 2 B | name | data length 4 B | data ]
```

Part	Meaning
`XPK1`	"This is an xPython package, format 1"
manifest	JSON describing the app, including the **SHA-256 of every file**. This is the only signed part
signature	ECDSA P-256, raw `r||s` (32 + 32 bytes), over `b"xpy-app-v1" + manifest`
file entries	The app's files, usually `.mpy` bytecode (no source code on the device)

The manifest

The developer writes `app.json`; `xpy-pack build` compiles `.py` to `.mpy`, adds the hashes and signs. For `apps/sensor`:

```
{"app_id": "sensor", "version": "1.0.0", "entry": "main.mpy",
 "files": {"main.mpy": "9c1f…e3"},                      # SHA-256 of each file
 "permissions": {"publish": ["telemetry"], "time_ms": 15000}}
```

Field	Rule checked by xloader
`app_id`, file names	1–32 characters, no `/` or `\`, not starting with `.`, not `manifest.json` / `manifest.sig`
`version`	Exactly `x.y.z` (integers), and **newer** than the installed version
`entry`	Must be one of the files
`permissions.fs`	`{"read": [...], "write": [...]}`: absolute paths, no `..`
`permissions.keys`	xkeys labels the app may use
`permissions.modules`	Extra modules the app may import (forbidden ones stay forbidden)
`permissions.publish`	MQTT subtopics: no `+`, no `..`, `#` only at the end
`permissions.time_ms`	1–30000 ms; **5000 ms** if missing

> **Key idea: sign the manifest, hash the files.** The signature covers only the small manifest, but the manifest holds the hash of every file. Changing one byte of a file changes its hash, which no longer matches the signed manifest. This is a **hash list** (a one-level Merkle tree).

3. Main functions

Function	What it does	Returns
`install(path)`	Reads a `.xpk`, runs every check, writes the app to `/apps/<app_id>/`	app_id / `ValueError`
`verify(app)`	Re-checks an installed app: the stored signature, then every file hash	manifest dict / `ValueError`
`run(app)`	`verify()`, then sandbox + time limit, import the entry file, call `main()`, always clean up	what `main()` returns
`cleanup()`	Turns off sandbox, timer and `/data`. Safe to call many times	None
`list()`	Installed apps (hidden folders `.stage`, `.data` are skipped)	list of names
`info(app)`	The stored manifest, **without** checking it	dict / None
`remove(app)`	Deletes the app, its private data folder and its private data key	None
`set_root(path)`	Changes the install folder (PC tests use `/tmp/…`)	None

4. Data flow

Installing: a chain of checks

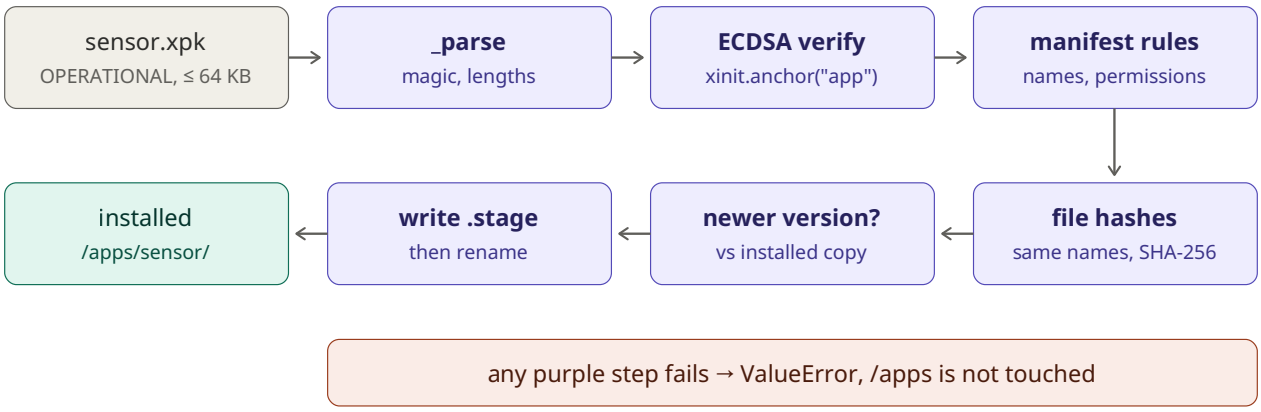


Figure 1. install(): nothing is written until every check has passed.

Check	Stops
Device is `OPERATIONAL`	Installing on a blank, unfinished or LOCKED device
Size ≤ 64 KB	Running out of RAM (the whole package is read into memory)
Signature with the app anchor	Packages not signed by your app developer key
Signature checked **before** `json.loads`	Untrusted JSON reaching the parser
Manifest rules	Path escapes like `../boot.py`, bad permissions
File hashes	Files changed, added or removed after signing
Newer version	Downgrade to an older (buggy) version, re-install of the same version
Write to `.stage`, then rename	A half-installed app: the folder is the old version or the new one, never a mix

The app folder also keeps `manifest.json` and `manifest.sig`, so the app can be checked again later.

Running: verify, sandbox, run, clean up

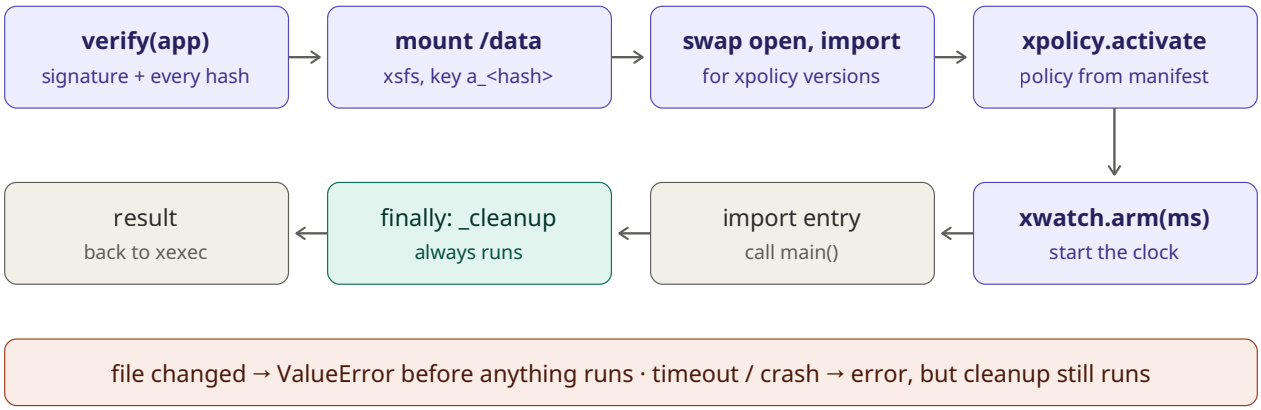


Figure 2. run(): the app is checked again, boxed in, timed, and always cleaned up.

Cleanup order: `xwatch.disarm()` → `xpolicy.deactivate()` → restore `open` and `__import__` → forget the app's modules and path → unmount `/data`. The timer goes **first**, so a timeout can't interrupt the cleanup halfway. `xexec` also calls `xloader.cleanup()` after every app, as a second safety net.

The policy that run() builds

For the sensor manifest, `_policy()` gives `xpolicy`:

```
{"app": "sensor",
 "modules": ["main", "xmsg"],     # own files + "modules" + xkeys/xmsg if needed
 "read":    [],
 "write":   ["/data"],             # always its private folder
 "keys":    ["a_<12 hex>"],        # always its private data key
 "publish": ["telemetry"]}
```

Kind of module	Examples	Can the app import it?
Safe defaults	`time, math, json, struct, re, hashlib, xcrypto`	Always
`os`	`os.listdir, os.remove`	Yes, but a **restricted os**: every path is checked against read / write
Others	`machine`	Only if listed in `permissions.modules`
Forbidden	`sys, builtins, xinit, xloader, xpolicy, network, socket`	Never, even if the manifest asks

5. Who uses xloader, and what it uses

Module	Link	For
`xinit` (Module 3)	`state()`, `anchor("app")`	Install only when OPERATIONAL; the app anchor decides which developer is trusted

`xcrypto` (Module 1)	`ecdsa_verify()`, `sha256()`	Signature and file hashes
`xkeys` (Module 2)	`exists()`, `delete()`	Each app's private key `a_<hash>`; deleting it on `remove()` makes the app's old data unreadable
`xsfs` (Module 4)	`mount("/data", …)`	Each app's own sealed folder `/apps/.data/<app>`
`xpolicy` (next)	`open`, `import_hook`, `activate()`	Enforces the sandbox that xloader describes
`xwatch` (next)	`arm()`, `disarm()`	Stops the app when its time is over
`xexec` (later)	calls `run()`, `cleanup()`, `list()`	Runs apps at boot, catches every error, logs the result
`xpy-pack` (PC tool)	same `.xpk` format	Builds and signs packages with `keys/app_sign.pem`

Why the label `"xpy-app-v1"`? Each signature in xPython starts with its own label (`xpy-devcert-v1` for the device certificate, `xpy-app-v1` for apps). A signature made for one purpose can never be reused for another. This is called **domain separation**.

6. Test it yourself

> **Where to test:** Steps 1–7 use the **PC build** (start the PC REPL with the MICROPYPATH command from CLAUDE.md). They need a freshly provisioned device, so **don't run them on board A**. Step 8 is read-only and safe on board A. For multi-line blocks, use paste mode: **Ctrl-E**, paste, **Ctrl-D**.

Step 1 · Provision the device in RAM (you play the vendor)

```
import xinit, xcrypto, xloader, json, struct
PIN = "123456"
xinit.set_pin(PIN)
dev_id, dev_pub = xinit.init(PIN)
v_priv, v_pub = xcrypto.ecdsa_keygen()
a_priv, a_pub = xcrypto.ecdsa_keygen()
xinit.install_anchor(PIN, "vendor", v_pub)
xinit.install_anchor(PIN, "app", a_pub)
xinit.install_anchor(PIN, "fw", xcrypto.ecdsa_keygen()[1])
cert = xcrypto.ecdsa_sign(v_priv, b"xpy-devcert-v1" + dev_id + dev_pub)
xinit.set_certificate(PIN, cert)
xinit.finalize(PIN)
xinit.state()                                   # 'OPERATIONAL'
```

Step 2 · A mini packer (paste mode): the same format as xpy-pack

```
def pack(ver, code, key=None):
    files = {"main.py": code}
    hashes = {n: xcrypto.sha256(d).hex() for n, d in files.items()}
    m = json.dumps({"app_id": "demo", "version": ver, "entry": "main.py",
                    "permissions": {}, "files": hashes}).encode()
    sig = xcrypto.ecdsa_sign(key or a_priv, b"xpy-app-v1" + m)
    blob = b"XPK1" + struct.pack("<I", len(m)) + m + sig
    for n, d in files.items():
        blob += struct.pack("<H", len(n)) + n.encode()
        blob += struct.pack("<I", len(d)) + d
    with open("/tmp/demo.xpk", "wb") as f:
        f.write(blob)
    return "/tmp/demo.xpk"
```

Step 3 · Install and run

```
xloader.set_root("/tmp/xl")
if "demo" in xloader.list(): xloader.remove("demo")   # leftovers from an earlier session
xloader.install(pack("1.0.0", b"def main():\n    return 42\n"))   # 'demo'
xloader.list()                                  # ['demo']
xloader.info("demo")["version"]                 # '1.0.0'
xloader.run("demo")                             # 42
```

> **Works when:** the app installs and `run()` returns 42.

Step 4 · Attack: wrong signer, same version

```
k, _ = xcrypto.ecdsa_keygen()
xloader.install(pack("1.1.0", b"x=1", k))       # ValueError: bad signature
xloader.install(pack("1.0.0", b"x=1"))          # ValueError: version must be newer than 1.0.0
```

Step 5 · Attack: change a file in flash after install

```
with open("/tmp/xl/demo/main.py", "ab") as f:
    f.write(b"\n# evil")
xloader.run("demo")                             # ValueError: app modified: main.py
```

Step 6 · Attack: leave the sandbox

```
xloader.install(pack("1.1.0", b"import xinit\ndef main(): return 1\n"))
xloader.run("demo")                             # ImportError: policy: module 'xinit' not allowed
xloader.install(pack("1.2.0", b"import os\ndef main(): return os.listdir('/')\n"))
xloader.run("demo")                             # OSError: policy: read '/' denied
```

Note that `import os` itself works: the app gets the restricted `os`, and the **read** is refused.

Step 7 · Attack: run forever, then check the cleanup

```
xloader.install(pack("1.3.0", b"def main():\n    while True: pass\n"))
xloader.run("demo")                                # KeyboardInterrupt after about 5 s
import os; os.listdir("/tmp/xl")                   # ['.data', 'demo']: no policy left active
xloader.remove("demo"); xloader.list()             # []
```

> **All attacks refused:** each line above must fail with the error shown. If one of them returns a value, the protection is broken.

Step 8 · Board A (read-only)

```
mpremote connect /dev/ttyUSB0 repl
import xloader
xloader.list()                                     # your installed apps, e.g. ['hello', 'sensor']
xloader.verify("hello")["files"]                   # {'main.mpy': '<sha256 hex>'}
xloader.run("hello")                               # prints Hello from a signed xPython app!  → 'ok'
```

If the REPL is locked by `xexec`, open the unlock window first with `xpy-officer unlock`. Building and re-installing `apps/hello` with `xpy-pack` answers version must be newer than 1.0.0: that is the downgrade check working. Raise the version in `app.json` to install it for real.

> **Verified:** Steps 1–7 were run on the PC build of your firmware; every result matched what is written above. Step 8 was not run.

7. Limits to state in your thesis

Limit	Why	Possible fix
Downgrade after remove: `remove()` then `install()` of an old signed version works	The version is only compared with the copy installed now	A per-app minimum version in NVS that survives `remove()`, or a vendor-signed revocation list
Install is **not power-safe** between the two renames	The app is only in `.old`, and the next install deletes `.old`	At boot: if `.old` exists and the app folder is missing, rename it back
The version check reads `manifest.json` **without verifying** it	`info()` doesn't check the signature	Use `verify(app)` to read the old version
Check-then-use gap: files are read again from flash at import, after `verify()`	Needs a physical attacker writing flash at that moment	Flash encryption, or import from the bytes already checked
Apps readable with a **flash dump**	No flash encryption; `.mpy` is easy to disassemble	Flash encryption (SBFE variant)
`remove()` needs **no authorization**	No PIN or signature; only the REPL lock protects it	Require the SO PIN or a signed remove command

Summary

Goal	Only signed, unmodified apps are installed and run, and only inside their declared sandbox
Mechanism	`.xpk` = XPK1 + manifest + ECDSA signature over `"xpy-app-v1"` + manifest + files; the manifest holds every file's SHA-256
Functions	`install()`, `verify()`, `run()`, `cleanup()`, `list()`, `info()`, `remove()`
Data flow	Install: parse → signature → manifest rules → hashes → newer version → stage + rename. Run: verify → /data → policy → timer → main() → cleanup
Uses / used by	Uses xinit, xcrypto, xkeys, xsfs, xpolicy, xwatch · used by xexec and xpy-pack
Doesn't stop	Downgrade after remove, power loss during install, a flash dump of the apps

Next: Module 6 · `xpolicy` + `xwatch`, the sandbox and the time limit.